Projekt: Data-Mart-Erstellung in SQL

Finalisierungsphase

IU Internationale Hochschule

Studiengang: Angewandte Künstliche Intelligenz

Bookshare

SQL-Dokumentation

Verfasser	Rüdiger Schlenter
Matrikelnummer	IU14092699
Tutorin	Anna Androvitsanea
Eingereicht	25.06.2026

Abbildungsverzeichnis 3
1. Zweck des Dokuments 4
2. Ausführungsreihenfolge der SQL-Skripte 4
3. Testdaten 4
4. Abfragen für Sonderfälle 6
4.1. Eine Zeile je Buch in der Liste aller Bücher 6
4.2. Genre-Verteilung der Bücher 7
4.3. Überfällige Ausleihen 8
4.4. Exemplare ohne Ausleihanfrage 8
4.5. Unbeantwortete Ausleihanfragen 9
4.6. Abweichung bei der Rückgabe 10
5. Stored Procedures, Trigger, Views, Indizes 10
5.1. Geschäftslogik auf dem Datenbankserver 11
5.2. Kopplung der Anwendungsmodule 11
5.3. Indizes in Oracle-Datenbanken 12

Abbildungsverzeichnis

Abbildung 1: Liste aller Bücher, eine Zeile je Buch 7
Abbildung 2: Anzahl der Bücher pro Genre 7
Abbildung 3: Überfällige Ausleihen 8
Abbildung 4: Exemplare ohne Ausleihanfrage 9
Abbildung 5: Anfragen ohne Antwort 9
Abbildung 6: Abweichung bei der Rückgabe 10
Abbildung 7: Trigger übernimmt Geschäftslogik 11
Abbildung 8: View im Katalog-Modul 12
Abbildung 9: Sinnvolle Indizes 13

1. Zweck des Dokuments

Die Bookshare-Datenbank kann mithilfe von SQL-Skripten aufgesetzt werden, die dem Abgabepaket beiliegen. Sie werden in diesem Dokument beschrieben. Des Weiteren behandelt das Dokument die Testdaten, ihre Herkunft, ihre Aufbereitung und den Import in die Datenbank. Es enthält Abfragen mit ihren Ergebnissen und ordnet Stored Procedures, Trigger, Views und Indizes ein. Anders als die Phase-2-Abgabe, die je Entität einen Testfall zeigte, hält dieses Dokument zu den Abfragen das erwartete und das tatsächliche Ergebnis fest.

2. Ausführungsreihenfolge der SQL-Skripte

Die vier beigelegten Skripte laufen in der Reihenfolge, die sich aus den Abhängigkeiten zwischen Schema, Tabellen und Daten ergibt.

`create_user.sql` legt das Schema an, unter dem die Tabellen entstehen. Diese Methode ist optional, das Schema lässt sich auch über die Oberfläche eines Datenbank-Clients anlegen. In beiden Fällen genügt das Oracle-Systemprivileg CREATE USER.

`create_tables.sql` erzeugt die insgesamt sechzehn Tabellen des Bookshare-Schemas samt Primär- und Fremdschlüsseln und läuft vor dem Einspielen der Daten. Diese Reihenfolge ist zwingend, da eine INSERT-Anweisung eine bestehende Tabelle braucht.

`dummy_data.sql` füllt die Tabellen mit den Testdaten. Die Reihenfolge der INSERT-Anweisungen folgt den Fremdschlüsseln, also zuerst die referenzierten, dann die referenzierenden Tabellen. Am Anfang des Skripts steht der SQL*Plus-Befehl SET DEFINE OFF. Ohne ihn deutet der Datenbank-Client das Kaufmanns-Und („&“) in Buchtiteln als Substitutionsvariable und verlangt beim Lauf des Skripts eine Eingabe.

`queries.sql` enthält Abfragen gegen ein mit Daten gefülltes Datenbankschema. Diese Abfragen können einzeln ausgeführt werden, um Auswertungen durchzuführen, aber prinzipiell kann das Skript auch als Block laufen.

3. Testdaten

Die verwendeten Testdaten enthalten echte Buchdaten aus externen Quellen sowie fiktive Daten zu Nutzern, Exemplaren und Verleihvorgängen.

Titel, Sprachen, Erscheinungsjahre und Autoren wurden aus der Open Library Search API abgerufen, die Herkunftsländer der Autoren stammen aus Wikidata. Die Verbindung beider Quellen läuft über die Open-Library-Autoren-ID, die in Wikidata als Property P648 hinterlegt ist. Ein gebündelter Aufruf holt diese Länder für alle Autoren auf einmal.

Open Library liefert zu jedem Buch eine Reihe von Schlagwörtern. Sie werden mit einer festen Liste von Genres abgeglichen und so jedem Buch zugeordnet. Der Abgleich prüft hierbei ganze Wörter, weil eine Suche nach Teilzeichenketten in den Schlagwörtern fast alle Bücher falsch einordnet.

Auch bei den erfundenen Daten musste an zahlreichen Stellen auf fachliche Stimmigkeit geachtet werden, nicht nur auf gültige Fremdschlüssel:

- Jedes Exemplar gehört einem Nutzer, und sein Abholort ist eine Adresse genau dieses Besitzers.
- Eine Ausleihanfrage richtet sich an den Besitzer des Exemplars, und Leiher und Verleiher sind verschieden.
- Der Anfragetext entspricht der ersten Chat-Nachricht zum selben Vorgang.
- Eine Ausleihe entsteht nur aus einer bestätigten Anfrage, und ihr Datum liegt nach dem Anfragedatum.
- Der Ausleihstatus passt zu den Datumsangaben: abgeschlossen mit Rückgabedatum, abgebrochen mit Abbrecher und Grund.
- Eine Rückgabe gehört zu einer Ausleihe, betrifft dasselbe Exemplar und enthält deren tatsächliches Rückgabedatum.
- Nachrichten stammen abwechselnd von Leiher oder Verleiher und sind zeitlich geordnet.
- Eine Bewertung hängt an einer abgeschlossenen Ausleihe, stammt von einem Beteiligten und bewertet entweder das Buch oder den Partner, nicht beides.
- Bei Duplikatmeldungen sind Original und Duplikat verschiedene Bücher, und der Bearbeiter hat die Pfleger-Rolle.
- Offene Meldungen haben keinen Bearbeiter und kein Bearbeitungsdatum, erledigte beides, mit Bearbeitung nach Meldung.

Der Datenbestand verteilt sich wie folgt auf die sechzehn Tabellen:

Tabelle	**Kategorie**	**Zeilen**
buch_genre	Echte Buchdaten	678
buch_autor	Echte Buchdaten	335
buch	Echte Buchdaten	300
autor	Echte Buchdaten	166
genre	Feste Werte	13
exemplar	Erfundene Daten	176
adresse	Erfundene Daten	52
nutzer_rolle	Erfundene Daten	32

nutzer	Erfundene Daten	30
ausleihanfrage	Erfundene Daten	19
anfrage_nachricht	Erfundene Daten	15
ausleihe	Erfundene Daten	12
rueckgabe	Erfundene Daten	12
bewertung	Erfundene Daten	12
duplikat_meldung	Erfundene Daten	10
rolle	Feste Werte	4

Tabelle 1: Datenbestand

4. Abfragen für Sonderfälle

In Anwendungen, in denen relationale Datenbanken eingesetzt werden, enthält der Großteil der Abfragen keine oder wenige Joins und wenige Filter. Die in der Datei `queries.sql` gezeigten Datenbankoperationen sind meist komplexer und zeigen unter anderem Sonderfälle im Bookshare-Kontext: Die Resultate werden erst durch Zählen, Vergleichen über mehrere Vorgänge oder das Suchen nach Fehlendem ermittelt.

Die Datei `queries.sql` enthält weitere Abfragen dieser Art; die folgenden Abschnitte erläutern eine Auswahl.

4.1. Eine Zeile je Buch in der Liste aller Bücher

Eine Katalogübersicht listet alle Bücher mit je einem Autor und einem Genre. Ein einfacher Join würde ein Buch mit mehreren Autoren oder Genres mehrfach aufführen, deshalb soll im Abfrageergebnis jedes Buch auf eine Zeile beschränkt werden.

Die vier Joins erzeugen je Kombination aus Autor und Genre eine eigene Zeile, ein Buch mit drei Autoren und zwei Genres also sechs. Zwei Unterabfragen in der WHERE-Clause greifen je Buch die kleinste `autor_id` und die kleinste `genre_id` heraus, verbunden über die `buch_id` der äußeren Abfrage. So bleibt je Buch die Kombination aus erstem Autor und erstem Genre übrig.

Erwartet werden 300 Zeilen, eine je Buch, mit Titel, Autor, Genre, Sprache und Erscheinungsjahr, alphabetisch nach Titel sortiert.

```sql
SELECT  b.titel,
    a.vorname || ' ' || a.nachname  AS autor,
    g.bezeichnung           AS genre,
    b.sprache,
    b.erscheinungsjahr
FROM    buch b
JOIN    buch_autor ba   ON ba.buch_id  = b.buch_id
JOIN    autor a     ON a.autor_id  = ba.autor_id
JOIN    buch_genre bg   ON bg.buch_id  = b.buch_id
JOIN    genre g     ON g.genre_id  = bg.genre_id
WHERE   ba.autor_id = (
    SELECT  MIN(ba2.autor_id)
    FROM    buch_autor ba2
    WHERE   ba2.buch_id = b.buch_id
)
AND bg.genre_id = (
    SELECT  MIN(bg2.genre_id)
    FROM    buch_genre bg2
    WHERE   bg2.buch_id = b.buch_id
)
ORDER BY    b.titel;
```

bfrageergebnis × | Abfrageergebnis 1 ×

SQL | Alle Zeilen abgerufen: 300 in 0,06 Sekunden

Abbildung 1: Liste aller Bücher, eine Zeile je Buch

4.2. Genre-Verteilung der Bücher

Bei der Datenaufbereitung erhält jedes Buch anhand seiner Schlagwörter automatisch ein oder mehrere Genres. Diese Abfrage zählt je Genre die zugeordneten Bücher. Der LEFT JOIN geht von der Genre-Tabelle aus, sodass ein Genre ohne zugeordnetes Buch sichtbar bleibt. COUNT zählt dabei die Werte der Spalte `buch_id`; ein leeres Genre erscheint also mit null. COUNT(*) ergäbe stattdessen eins, weil es die vom LEFT JOIN erzeugte Zeile auch ohne Buchbezug mitzählt.

Erwartet wird je Genre eine Zeile mit der Menge der Bücher dieses Genres.

```sql
SELECT g.bezeichnung, COUNT(bg.buch_id) AS anzahl_buecher
FROM genre g
LEFT JOIN buch_genre bg ON bg.genre_id = g.genre_id
GROUP BY g.bezeichnung
ORDER BY anzahl_buecher DESC;
```

bfrageergebnis ×

SQL | Alle Zeilen abgerufen: 13 in 0,071 Sekunden

	BEZEICHNUNG	ANZAHL_BUECHER
1	Fantasy	151
2	Science Fiction	149
3	Adventure	83
4	Children	75
5	Mystery	48
6	Romance	39
7	History	36
8	Horror	32
9	Classics	25
10	Young Adult	23
11	Thriller	13
12	Sonstiges	3
13	Biography	1

Abbildung 2: Anzahl der Bücher pro Genre

4.3. Überfällige Ausleihen

Um säumige Leiher anzumahnen, müssen zuerst die überfälligen Ausleihen ermittelt werden. Überfällig ist eine laufende, also noch nicht zurückgegebene Ausleihe, sobald ihr geplantes Rückgabedatum verstrichen ist. Für jede überfällige Ausleihe zählt die Abfrage die Tage der Überschreitung.

Erwartet wird je überfälliger Ausleihe eine Zeile mit der Dauer der Überschreitung, absteigend nach dieser Dauer sortiert. In den Testdaten überschreitet nur eine Ausleihe das geplante Rückgabedatum, das Ergebnis enthält also eine Zeile. Diese Dauer erscheint als Bruchzahl, weil das aktuelle Datum auch die Uhrzeit enthält, und ergibt erst nach Abschneiden der Nachkommastelle ganze Tage.

```
-- Ausleihe
SELECT a.ausleihe_id, b.titel,
       n.vorname || ' ' || n.nachname AS leiher,
       a.geplantes_rueckgabedatum,
       SYSDATE - a.geplantes_rueckgabedatum AS tage_ueberfaellig
FROM ausleihe a
JOIN ausleihanfrage an ON an.anfrage_id = a.anfrage_id
JOIN exemplar e ON e.exemplar_id = an.exemplar_id
JOIN buch b ON b.buch_id = e.buch_id
JOIN nutzer n ON n.nutzer_id = an.leiher_id
WHERE a.status = 'LAUFEND'
AND a.geplantes_rueckgabedatum < SYSDATE
ORDER BY tage_ueberfaellig DESC;
```

Alle Zeilen abgerufen: 1 in 0,009 Sekunden

AUSLEIHE_ID	TITEL	LEIHER	GEPLANTES_RUECKGABEDATUM	TAGE_UEBERFAELLIG
3007	The Hero of Ages	Maria Bauer	20.05.26	35,802893518518518518518518518518518518 52

Abbildung 3: Überfällige Ausleihen

4.4. Exemplare ohne Ausleihanfrage

Ein Teil der eingestellten Exemplare wird nie angefragt. Diese Auswertung findet sie, damit die App sie auf der Startseite hervorheben oder den Verleiher per Mail darauf hinweisen kann.

Über das NOT EXISTS in der Abfrage bleiben nur die Exemplare ohne zugehörige Ausleihanfrage übrig. NOT EXISTS funktioniert hier im Gegensatz zu NOT IN. Sobald die Unterabfrage einen NULL-Wert enthält, liefert NOT IN kein Ergebnis.

Erwartet wird je Exemplar ohne Ausleihanfrage eine Zeile mit Titel und Besitzer, nach Titel sortiert. Da bisher nur wenige Exemplare angefragt wurden, enthält das Ergebnis den Großteil des Bestands.

Arbeitsblatt | Query Builder

```
SELECT  b.titel,
        n.vorname || ' ' || n.nachname AS besitzer
FROM    exemplar e
JOIN    buch b   ON b.buch_id   = e.buch_id
JOIN    nutzer n ON n.nutzer_id = e.besitzer_id
WHERE   NOT EXISTS (
            SELECT 1
            FROM   ausleihanfrage a
            WHERE  a.exemplar_id = e.exemplar_id
        )
ORDER BY b.titel;
```

Abfrageergebnis

SQL | Alle Zeilen abgerufen: 161 in 0,066 Sekunden

	TITEL	BESITZER
1	Записки изъ подполья	Nico Richter
2	Мастер и Маргарита	Jan Hoffmann
3	Мастер и Маргарита	Sara Schwarz

Abbildung 4: Exemplare ohne Ausleihanfrage

4.5. Unbeantwortete Ausleihanfragen

Stellt ein Leiher eine Anfrage, ohne dass der Verleiher antwortet, bleibt der Vorgang offen. Diese Auswertung findet die Anfragen, auf die der Verleiher seit mehr als sieben Tagen nicht reagiert hat, und liefert der App den Auslöser, ihn zu erinnern oder die Anfrage nach einer Frist zu schließen. Über NOT EXISTS bleiben nur die Anfragen ohne eine Nachricht des Verleihers übrig.

Erwartet wird je unbeantworteter Anfrage eine Zeile mit den Beteiligten und der Zahl der offenen Tage, absteigend nach dieser Zahl sortiert.

```
SELECT  an.anfrage_id,
        an.anfragedatum,
        l.vorname || ' ' || l.nachname AS leiher,
        v.vorname || ' ' || v.nachname AS verleiher,
        an.nachricht,
        TRUNC(SYSDATE - an.anfragedatum) AS tage_offen
FROM    ausleihanfrage an
JOIN    nutzer l ON l.nutzer_id = an.leiher_id
JOIN    nutzer v ON v.nutzer_id = an.verleiher_id
WHERE   SYSDATE - an.anfragedatum > 7
AND     NOT EXISTS (
            SELECT 1
            FROM   anfrage_nachricht m
            WHERE  m.anfrage_id = an.anfrage_id
            AND    m.sender_id  = an.verleiher_id
        )
ORDER BY tage_offen DESC;
```

bfrageergebnis | Abfrageergebnis 1

SQL | Alle Zeilen abgerufen: 13 in 0,008 Sekunden

	ANFRAGE_ID	ANFRAGEDATUM	LEIHER	VERLEIHER	NACHRICHT	TAGE_OFFEN
1	2017	05.01.26	Thomas Zimmermann	Olga Klein	(null)	170
2	2018	08.01.26	Viktor Krüger	Sara Schwarz	(null)	167
3	2001	15.01.26	Eva Weber	Clara Schneider	Würde das Buch gerne ausleihen.	160
4	2002	22.01.26	Greta Wagner	Felix Meyer	(null)	153
5	2004	05.02.26	Lars Koch	Karin Schäfer	(null)	139

Abbildung 5: Anfragen ohne Antwort

4.6. Abweichung bei der Rückgabe

Werden Bücher im Schnitt deutlich zu spät zurückgegeben, halten sich viele Leiher nicht an die vereinbarte Frist. Diese Auswertung bildet je abgeschlossener Ausleihe die Differenz zwischen tatsächlichem und geplantem Rückgabedatum und mittelt sie über alle Vorgänge. Bei einer hohen mittleren Verspätung ließe sich der Bedarf für Erinnerungen oder Mahnungen ableiten. Eine frühere Rückgabe zählt negativ, eine spätere positiv.

Der innere JOIN von `ausleihe` auf `rueckgabe` filtert implizit auf abgeschlossene Ausleihen; AVG und ROUND berechnen die mittlere Abweichung.

Erwartet wird eine einzige Zahl, der Mittelwert der Abweichung über alle abgeschlossenen Ausleihen.

```
SELECT ROUND(AVG(r.rueckgabedatum - a.geplantes_rueckgabedatum), 1)
       AS mittlere_abweichung_tage
FROM   ausleihe a
JOIN   rueckgabe r ON r.ausleihe_id = a.ausleihe_id;
```

ageergebnis × | Abfrageergebnis 1 ×

SQL | Alle Zeilen abgerufen: 1 in 0,012 Sekunden

MITTLERE_ABWEICHUNG_TAGE
-3,4

Abbildung 6: Abweichung bei der Rückgabe

5. Stored Procedures, Trigger, Views, Indizes

Eine Datenbank speichert Daten und sichert ihre Integrität. Wie viel Verantwortung die Datenbank außerdem haben sollte, ist eine Frage von Architekturprinzipien. Zieht man die Grenze zwischen Datenbank und Anwendung eng, dann werden Entscheidungen über Abläufe der Anwendung überlassen.

Ein weiterer Grundsatz teilt Anwendungen in Module mit eigenen Grenzen auf. Ein Datenbankobjekt, das Tabellen mehrerer Module verbindet, verletzt diese Grenzen.

An diesen Prinzipien lassen sich Stored Procedure, Trigger und View einordnen. Eine Stored Procedure und ein Trigger verschieben Geschäftslogik auf den Datenbankserver. Eine View und ebenfalls die Stored Procedure können Tabellen über Modulgrenzen hinweg verbinden.

Ob der Einsatz der genannten Objekte vertretbar ist, hängt vom Einzelfall ab und verlangt Augenmaß. Ein Index hingegen wirft diese Frage nicht auf, weil er weder Logik ausführt noch Module verbindet, sondern allein die Zugriffsgeschwindigkeit betrifft.

5.1. Geschäftslogik auf dem Datenbankserver

Ein Bookshare-Ausleihprozess durchläuft mehrere Status. Bestätigt ein Verleiher eine Anfrage, wird aus der Anfrage eine Ausleihe. Mit der Rückgabe wechselt deren Status auf abgeschlossen. Beide Zustandsübergänge ließen sich auf den Datenbankserver verlagern.

Eine Stored Procedure könnte den ersten Übergang in einem Aufruf bündeln. Sie setzte den Status der Ausleihanfrage auf BESTAETIGT, legte den passenden Datensatz in `ausleihe` an und gäbe dessen Kennung zurück. Ein Trigger auf `rueckgabe` übernähme den zweiten Übergang und setzte beim Einfügen einer Rückgabe den Status der zugehörigen Ausleihe selbsttätig auf ABGESCHLOSSEN.

Geplant ist, dass diese Statuswechsel in der Anwendung durchgeführt werden. Der Datenbankserver übernimmt keine Verantwortung für diesen Prozess.

```
CREATE OR REPLACE TRIGGER rueckgabe_schliesst_ausleihe
AFTER INSERT ON rueckgabe
FOR EACH ROW
BEGIN
    UPDATE ausleihe
       SET status = 'ABGESCHLOSSEN',
           tatsaechliches_rueckgabedatum = :new.rueckgabedatum
     WHERE ausleihe_id = :new.ausleihe_id;
END;
/
rollback;
```

Skriptausgabe

Aufgabe abgeschlossen in 0,039 Sekunden

```
Trigger RUECKGABE_SCHLIESST_AUSLEIHE kompiliert

Rollback abgeschlossen.
```

Abbildung 7: Trigger übernimmt Geschäftslogik

5.2. Kopplung der Anwendungsmodule

Das Bookshare-Schema gliedert sich in vier Module: Katalog, Ausleihprozess, Benutzerverwaltung und Bewertungen. Ein Datenbankobjekt, das Tabellen mehrerer Module verbindet, schafft eine Abhängigkeit zwischen ihnen per SQL.

Eine View über die nie angefragten Exemplare zeigt das. Für Titel und Besitzer verknüpfte sie `exemplar` aus dem Ausleihprozess, `buch` aus dem Katalog und `nutzer` aus der Benutzerverwaltung. Drei Module wären damit über ein einziges Objekt aneinander gebunden.

Das gleiche Problem existiert in der im letzten Abschnitt skizzierten Stored Procedure, die `nutzer` und `exemplar` liest, während sie in `ausleihe` schreibt. In einer modularisierten Anwendung umgeht eine solche Verbindung die Modulgrenze.

Unkritisch bleibt eine View, solange sie Tabellen innerhalb eines Moduls verknüpft. Die Katalogübersicht aus Kapitel 4 verbindet `buch`, `autor` und `genre`, also ausschließlich Katalogtabellen.

```
CREATE OR REPLACE VIEW v_buchliste AS
SELECT b.titel,
       a.vorname,
       a.nachname,
       g.bezeichnung AS genre,
       b.sprache,
       b.erscheinungsjahr
  FROM buch b
  JOIN buch_autor ba ON ba.buch_id = b.buch_id
  JOIN autor      a  ON a.autor_id = ba.autor_id
  JOIN buch_genre bg ON bg.buch_id = b.buch_id
  JOIN genre      g  ON g.genre_id = bg.genre_id
 WHERE ba.autor_id = (SELECT MIN(ba2.autor_id)
                        FROM buch_autor ba2
                       WHERE ba2.buch_id = b.buch_id)
   AND bg.genre_id = (SELECT MIN(bg2.genre_id)
                        FROM buch_genre bg2
                       WHERE bg2.buch_id = b.buch_id)
 ORDER BY b.titel;
```

Skriptausgabe

Aufgabe abgeschlossen in 0,158 Sekunden

```
View V_BUCHLISTE erstellt.
```

Abbildung 8: View im Katalog-Modul

5.3. Indizes in Oracle-Datenbanken

Oracle legt für jeden Primärschlüssel und jede UNIQUE-Bedingung automatisch einen Index an. Suchen über diese Spalten sind damit schon abgedeckt. Das gilt beispielsweise für `anfrage_id` in `ausleihe`, weil diese Spalte UNIQUE ist.

Oracle kann zudem eigenständig Indizes anlegen. Dazu beobachtet die Datenbank, welche Spalten oft in Abfragen vorkommen. Diese Funktion heißt Automatic Indexing und es gibt sie nur für die Enterprise Edition auf Exadata und für die Autonomous Database. Auf einer normalen Installation wird sie nicht unterstützt.

Die eigentlichen Kandidaten für einen Index sind die nicht-eindeutigen Fremdschlüssel. Über sie laufen viele Joins und Filter, etwa `buch_id` in `exemplar` oder `ausleihe_id` in `bewertung`. Ein Index auf so einer Spalte hilft nicht nur beim Lesen. Ohne ihn sperrt Oracle beim Löschen einer Elternzeile die ganze Kindtabelle. Löscht man ein Buch, ist das die Tabelle `exemplar`.

Bei ein paar hundert Zeilen pro Tabelle bringt ein Index jedoch wenig. Oracle liest dann ohnehin die ganze Tabelle, da es schneller ist als der Umweg über einen Index. Erst bei deutlich mehr Daten lohnt er sich also.

Bei Geokoordinaten hilft ein normaler Index nicht. Die Distanzsuche rechnet mit der Haversine-Formel über `breitengrad` und `laengengrad`. Ein Index sortiert aber nur einzelne Spalten. Für

eine Umkreissuche muss man die Treffer zuerst über eine Bounding Box eingrenzen. Dann greift auch ein Index. Mit Oracle Spatial ginge das über einen räumlichen Index.

```
CREATE INDEX ix_exemplar_buch     ON exemplar  (buch_id);
CREATE INDEX ix_exemplar_besitzer ON exemplar  (besitzer_id);
CREATE INDEX ix_bewertung_ausleihe ON bewertung (ausleihe_id);
```

Skriptausgabe

Aufgabe abgeschlossen in 0,064 Sekunden

```
Index IX_EXEMPLAR_BUCH erstellt.

Index IX_EXEMPLAR_BESITZER erstellt.

Index IX_BEWERTUNG_AUSLEIHE erstellt.
```

Abbildung 9: Sinnvolle Indizes